

From Semantics to Execution: Dynamic Geometric Constraints for Tool-Action Feasibility

Anonymous submission

Abstract

A tool call can be valid and relevant to the task yet still be induced by untrusted content and cause an unauthorized external effect. Execution therefore requires evidence that grounds the proposed effect, establishes its authorization, and matches the current state. We present DAFR, a method for judging tool-action feasibility with dynamic geometric constraints. DAFR deterministically maps each tool call and its supporting evidence to interpretable action–evidence coordinates, then instantiates state-dependent constraints whose intersection forms the feasible region. Calls inside the region are executed; calls outside it are blocked or returned with targeted feedback identifying the unsupported execution relation. DAFR reconstructs the representation and active constraints as evidence and environment state change. On AgentDojo, DAFR reaches 0% observed attack success with 87.5% clean utility and 81.4% attack utility. On Agent Security Benchmark, it reaches 0% observed attack success with 86.8% utility and 73.5% task success. Across both benchmarks, the dynamic feasible region blocks observed attack-induced actions while retaining task progress.

Introduction

Tool-using agents can read private information, modify shared resources, contact external parties, and trigger actions that are difficult to reverse. These calls have more direct consequences than generated text: an executed call may immediately disclose information, change system state, or start an external operation (Yao et al. 2023; Schick et al. 2023; Ruan et al. 2024; Ye et al. 2024; Lee et al. 2026). Safe execution requires more than a valid schema. The function, effect-bearing arguments, expected effect, and state dependencies must be supported by the current task, available evidence, authorization, and policy.

Indirect prompt injection makes this decision difficult (Greshake et al. 2023; Liu et al. 2023; Zhan et al. 2024). An agent may need untrusted content to complete a valid task, yet that content can contain attacker-inserted instructions. The model can translate them into a well-formed, apparently relevant call that discloses sensitive information, modifies

the wrong object, or creates an unauthorized external effect (Debenedetti et al. 2024; Liao et al. 2025; Wang et al. 2025; Li et al. 2026; Ma et al. 2026). Tool descriptions and environment metadata provide additional channels for malicious control (Wang et al. 2026). The execution-time question is whether trusted evidence supports the call’s effect in the current state.

Consider an agent asked to update a customer record whose identifier is initially unknown. The agent retrieves the record, obtains its identifier from the verified result, and uses that identifier for the requested update. The retrieved content may also contain an attacker-issued command to send the record to a newly specified address. Both actions are schema-valid, but their support differs. The update follows the user request and uses an identifier established by the verified retrieval. The disclosure relies on a destination introduced through untrusted content and produces an unauthorized external effect. The execution layer must distinguish these actions by the evidence supporting their concrete effects.

Existing defenses act at different stages of the agent pipeline. Context-level methods separate trusted instructions from untrusted data, restructure prompts, repeat the user request, or filter suspicious observations before action generation (Hines et al. 2024; Wallace et al. 2024; Jia et al. 2025; Chen et al. 2025). Runtime methods apply symbolic policies, privilege checks, execution plans, or information-flow rules to the generated action (Wang, Poskitt, and Sun 2025; Shi et al. 2025; Li et al. 2025a; Debenedetti et al. 2025). Tool-action feasibility, however, often depends on relations among the action, its evidence, and the current state. A destination can be valid when specified by the user and invalid when copied from a web page. A write can become feasible after a verified read establishes its target. Combining sensitive data with an external destination can require stronger grounding and authorization than either factor alone.

We introduce DAFR (Dynamic Action Feasible Regions), a geometric method for deciding whether a concrete tool action is feasible at the current execution step. A deterministic semantic lifting function represents the call and its supporting evidence as interpretable action–evidence coordinates. Policy-specified halfspaces and joint-risk regions then encode the execution requirements active in the current state. Their intersection defines the feasible region: a call executes

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

inside the region, and a call outside it is blocked or returned with feedback identifying the unsupported relation.

The geometry changes with execution. A verified read can establish an identifier required by a later write; a trusted confirmation can authorize a specific external effect; and a new observation can revise an earlier state assumption. DAFR reconstructs the action–evidence point and active geometry before every external effect. Shared execution roles extend the same decision procedure across tools with different schemas but comparable effects.

We evaluate DAFR on AgentDojo with three planner models and on Agent Security Benchmark (ASB) with DeepSeek V4 Flash. DAFR reaches 0% observed attack success on both benchmarks, with 87.5% clean utility and 81.4% attack utility on AgentDojo and 86.8% utility and 73.5% task success on ASB. Removing geometric constraints raises attack success from 0% to 16.7%, and signed-margin analysis links each rejected call to the execution relation that limits feasibility. The consistent results across planners and benchmarks support DAFR as a unified execution-layer mechanism for reliable tool use.

DAFR: Dynamic Action Feasible Regions

DAFR constructs an interpretable action–evidence point for the proposed call and instantiates the geometric constraints active in the current state. Region membership determines feasibility, and signed margins identify the relations that limit the decision.

Problem Setup

At step t , a tool-using agent receives a trusted user task u , operates under an execution policy p , observes the environment, and produces a concrete tool call

$$a_t = (f_t, \rho_t), \quad (1)$$

where f_t is a registered tool and ρ_t maps the fields of the tool schema to argument values. The call is first checked against the schema. Passing this check establishes that the call is well formed and leaves its execution feasibility to the evidence and policy checks below.

Let E_t denote the execution state before a_t is evaluated. It contains the trusted task, execution policy, available tool schemas, established facts, completed prerequisites, authorizations, confirmations, and observations with source labels. Facts derived from external observations retain their source labels, while intent, confirmation, and authorization are recorded through trusted channels.

A call is feasible when the current evidence and policy support its function, the arguments that determine its external effect, the expected effect itself, and the required state conditions. We write

$$\text{Feas}(a_t \mid E_t, u, p) = 1 \quad (2)$$

when these conditions are satisfied. A feasible call is sent to the trusted runtime. An infeasible call is blocked or returned with feedback that identifies the function, argument, evidence source, authorization condition, or state assumption

requiring revision. After an admitted call returns observation o_{t+1} , the state is updated as

$$E_{t+1} = T(E_t, a_t, o_{t+1}), \quad (3)$$

where T may add evidence, satisfy a prerequisite, record authorization, or invalidate an earlier assumption.

Action–Evidence Point Construction

DAFR represents each proposed tool call and its current evidence as a point in an interpretable action–evidence space. The coordinates cover task support and progress, argument grounding and provenance, authorization and scope, and expected effects and risks.

DAFR uses a deterministic semantic lifting function to construct the action–evidence point:

$$z_t = L(a_t, E_t, u, p) = (q_t, g_t, c_t, r_t) \in [0, 1]^d, \quad (4)$$

where L deterministically evaluates explicit execution relations, and q_t , g_t , c_t , and r_t are the resulting coordinate blocks. The task block q_t records support for the selected function, its expected effect, and current execution progress. The grounding block g_t connects effect-bearing arguments to source-tagged evidence and established state. The authorization block c_t records confirmation, available authority, and the scope of the proposed effect. The risk block r_t describes consequences such as disclosure, transfer, deletion, redirection, and state change.

For each tool f , a role map

$$B_f : \text{Fields}(f) \rightarrow \mathcal{U} \quad (5)$$

maps tool-specific fields to execution roles in $\mathcal{U}$, including destination, object, data, amount, time, and effect. Recipients, payment accounts, and sharing targets map to the destination role; records, files, messages, and transaction details map to data or object roles; and reads, writes, disclosures, transfers, and deletions map to effect roles. The original field and value are retained so that feedback can name the exact argument that requires revision.

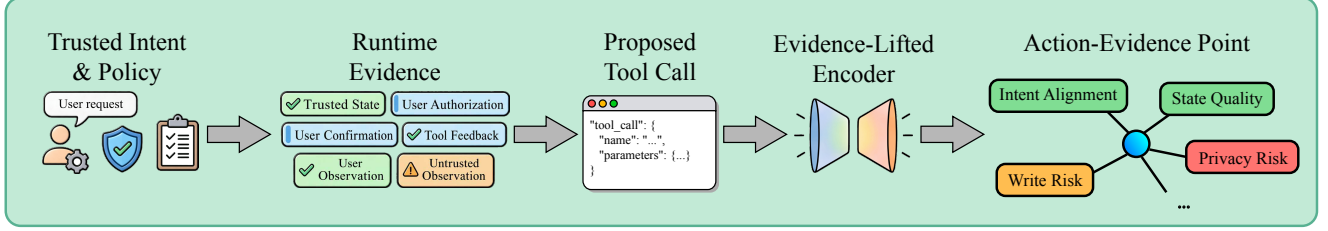
The evidence used by these relations is maintained in the execution state. After an admitted call returns o_{t+1} , an evidence projection operator converts it into source-tagged factual and risk records:

$$P(o_{t+1}) = (S_{t+1}^+, R_{t+1}^+), \quad (6)$$

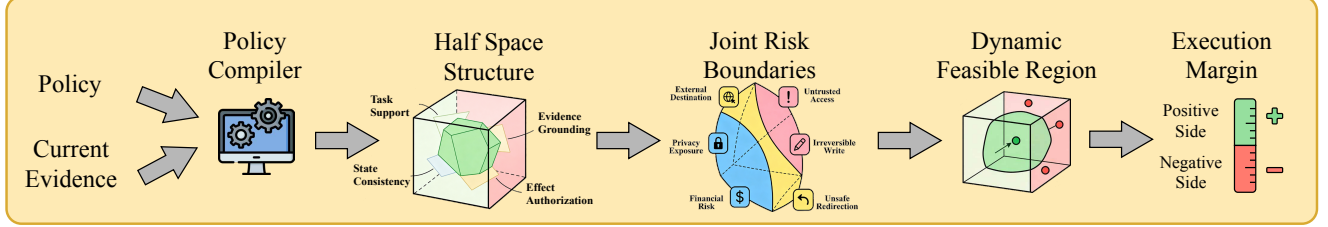
where S_{t+1}^+ contains entities, argument values, completed prerequisites, and observed state changes, and R_{t+1}^+ contains untrusted instructions, task conflicts, unsupported effect requests, and other risk signals. Each record retains its source, trust status, and execution role when incorporated into E_{t+1} . Factual records update grounding and state relations, risk records update source, conflict, and effect-risk relations, and trusted records establish intent, confirmation, and authorization.

Each coordinate has a defined execution meaning and takes a value in $[0, 1]$ under the current call and execution state. Coordinates for prerequisites and authorization record whether the corresponding conditions have been established, while

Step 1: Build the Action-and-Evidence Point



Step 2: Compile the Dynamic Feasible Region



Step 3: Decide, Execute, and Update

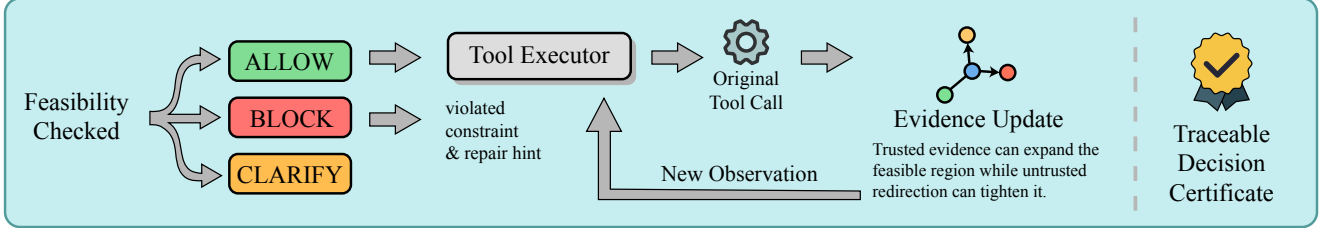


Figure 1: Overview of DAFR. A proposed tool call and its runtime evidence define an action–evidence point. State-conditioned halfspaces and joint-risk boundaries form the current feasible region. Region membership determines feasibility, while violated margins identify the limiting relations used for targeted revision or blocking. The resulting observation updates the next decision state.

grounding and scope coordinates record the support available for a specific argument or effect from source-tagged evidence. For example, a verified lookup can establish the identifier required by a later update, whereas a destination introduced by untrusted content receives no authorization support for an external disclosure.

For an effectful call, let $V(a_t)$ denote the argument values that directly determine its external effect, such as a destination, object identifier, amount, date, or sensitive data. DAFR evaluates each value against the trusted task, source-tagged evidence, and current state. The critical grounding score is

$$g_{\text{crit}}(a_t, E_t) = \min_{v \in V(a_t)} g(v, E_t), \quad (7)$$

where $g(v, E_t) \in [0, 1]$ measures the available support for value v . The minimum gives a conjunctive grounding summary across the arguments that determine the external effect: every critical value must satisfy its grounding requirement. Schema-optional fields follow the same effect-aware check. A field is eligible for omission during revision only when its removal leaves the destination, object, data, scope, authorization requirements, and expected external effect unchanged. Otherwise, an unsupported value keeps the call infeasible.

As evidence and execution state evolve, DAFR updates the

affected coordinates, so the same proposed call may occupy a different position at a later execution step.

Dynamic Geometric Constraints

DAFR converts the execution requirements at step t into geometric regions in the action–evidence space. Each policy defines fixed constraint templates and coefficients before evaluation; the same parameterization is used across all benchmark runs. For a proposed call, its expected effect, available evidence, authorization scope, policy, and current state select the active templates. Let $\mathcal{H}_t = \mathcal{H}(a_t, E_t, u, p)$ denote requirements expressed through individual execution relations. Each $j \in \mathcal{H}_t$ defines a halfspace

$$\mathcal{P}_j = \{z \in [0, 1]^d \mid m_j^{\text{half}}(z) = b_j - w_j^\top z \geq 0\}. \quad (8)$$

For each active requirement, the policy compiler retrieves w_j and b_j from the corresponding fixed template. The vector w_j selects and weights the relevant execution relations, and b_j sets the feasibility boundary. The coefficient signs follow the semantic direction of each relation: stronger trusted support raises the margin, and greater execution risk lowers it. These halfspaces represent task support, completed prerequisites, state consistency, critical-argument grounding, destination validity, confirmation, and authorization.

Some conditions depend on several relations at once. Let $\mathcal{K}_t = \mathcal{K}(a_t, E_t, u, p)$ denote the joint-risk conditions active at step t . Each $k \in \mathcal{K}_t$ defines a joint margin and its feasible region:

$$m_k^{\text{joint}}(z) = \beta_k + v_k^\top C_k z - \|W_k R_k z\|_2, \quad (9)$$

$$\mathcal{C}_k = \left\{ z \in [0, 1]^d \mid m_k^{\text{joint}}(z) \geq 0 \right\}.$$

The template for risk family k specifies the selectors C_k and R_k , the support weights v_k , the risk weights W_k , and the base risk budget β_k . The selectors extract the support and risk coordinates used by the condition. The support and risk weights are nonnegative, preserving the semantic direction of the joint margin. The curved boundary makes the acceptable level of each risk depend on the other risks present and the trusted support available for the action.

The feasible region is the intersection of all applicable regions:

$$\mathcal{F}_t = \left(\bigcap_{j \in \mathcal{H}_t} \mathcal{P}_j \right) \cap \left(\bigcap_{k \in \mathcal{K}_t} \mathcal{C}_k \right). \quad (10)$$

Only points that satisfy every relevant requirement belong to $\mathcal{F}_t$. DAFR instantiates this geometry at every step through the current z_t , $\mathcal{H}_t$, and $\mathcal{K}_t$. New evidence can move the point, introduce relevant boundaries, or satisfy conditions that previously prevented execution.

The running example shows how this geometry changes over a trajectory. Before the lookup returns, the update lacks the record identifier required by its grounding and prerequisite relations. The verified lookup establishes that value and allows the update point to enter the feasible region. The same observation provides no authority for an unrelated disclosure. That action combines an untrusted destination, sensitive data, and an external effect without matching authorization, so it remains outside $\mathcal{F}_t$. Evidence can therefore support one concrete effect while leaving another infeasible because the calls depend on different roles, sources, and authorization conditions.

Geometric Properties

Convexity and semantic direction. For a fixed execution context, every $\mathcal{P}_j$ is a halfspace. Each joint margin in Eq. (9) is concave because it is an affine function minus a norm, so its superlevel set $\mathcal{C}_k$ is convex. Their intersection $\mathcal{F}_t$ is therefore convex and defines one coherent feasible region for the current decision.

The parameter signs preserve a common semantic direction across the geometry. Trusted grounding, confirmation, and authorization increase the relevant margins, while externality, sensitivity, irreversibility, and untrusted control decrease them. Support and risk therefore move an action toward opposite sides of the active boundaries.

Joint risks. The curved regions capture how the acceptable level of one risk changes with the other risks present and with the trusted support available for the action. When multiple risk relations share one authorization budget, the norm term in Eq. (9) permits different allocations while limiting their

combined magnitude. An increase in one risk reduces the amount of the others that can be accepted under the same evidence and authorization.

Source-aware state changes. New records update the relations they establish. A verified read can increase grounding and prerequisite support for returned values; a trusted confirmation can increase scoped authorization for a specified effect, object, or destination; and an untrusted observation can contribute factual values while activating source, conflict, or redirection risks. These source-aware updates allow one observation to support a requested write while leaving an unrelated disclosure outside the feasible region.

Feasibility Decision and Targeted Feedback

Given z_t , DAFR evaluates every margin that defines $\mathcal{F}_t$. The overall feasibility margin is

$$M_t(z) = \min \left\{ \min_{j \in \mathcal{H}_t} m_j^{\text{half}}(z), \min_{k \in \mathcal{K}_t} m_k^{\text{joint}}(z) \right\}. \quad (11)$$

Because membership in $\mathcal{F}_t$ requires every active margin to be nonnegative,

$$z_t \in \mathcal{F}_t \iff M_t(z_t) \geq 0. \quad (12)$$

The sign of $M_t(z_t)$ is equivalent to region membership, and its magnitude records the raw signed slack under the most limiting active condition. DAFR executes the call when $M_t(z_t) \geq 0$. When an admissible revision path exists, a failed condition produces targeted feedback for the planner. All other infeasible calls are blocked.

A call may violate several boundaries. DAFR records

$$\mathcal{V}_t = \{c \in \mathcal{H}_t \cup \mathcal{K}_t \mid m_c(z_t) < 0\}, \quad (13)$$

where m_c denotes the halfspace or joint margin associated with condition c . For feedback ordering, DAFR places violated margins on comparable scales through $\bar{m}_c(z_t) = m_c(z_t)/\sigma_c$, with $\sigma_c > 0$. Raw margins continue to determine feasibility. The most limiting conditions are selected from $\mathcal{V}_t$, and $\phi_t = \Gamma(\mathcal{V}_t, z_t, E_t)$ maps them to concise revision targets. Feedback can identify an unsupported tool choice or argument, an unsuitable source, missing authorization, an incomplete prerequisite, an outdated state assumption, or the factors contributing to a joint-risk violation.

Each decision records the concrete call, its action-evidence point, the active boundaries, the margin vector, and the supporting evidence links. This record provides the same geometric basis for the execution decision and the revision target.

Runtime Update and Cost

Each invocation of DAFR evaluates one concrete proposed call. A feasible call is executed by the trusted runtime, and the returned observation is projected into source-tagged records and incorporated into the next execution state through Eqs. (6) and (3). An infeasible call produces no external effect. When targeted feedback is returned, the planner may revise the call, obtain additional evidence, request authorization, or end the task. Any subsequent proposal is evaluated as a new call under the state available at that step.

Algorithm 1 DAFR Execution-Step Decision

Input: proposed call $a_t = (f_t, \rho_t)$, state E_t , task u , policy p .**Output:** execution decision and feedback.

```

1: Check  $a_t$  against the registered schema of  $f_t$ .
2: Map tool fields to shared execution roles with  $B_{f_t}$ .
3: Build  $z_t = L(a_t, E_t, u, p)$  from the current relations.
4: Instantiate  $\mathcal{H}_t$  and  $\mathcal{K}_t$  from the active policy templates.
5: Evaluate all active margins and compute  $M_t(z_t)$ .
6: if  $M_t(z_t) \geq 0$  then
7:   Execute  $a_t$ , obtain  $o_{t+1}$ , and update  $E_{t+1}$ .
8: else
9:   Form  $\mathcal{V}_t$  and derive feedback  $\phi_t$ .
10:  if an admissible revision path exists then
11:    Return CLARIFY with  $\phi_t$  to the planner.
12:  else
13:    Return BLOCK.
14:  end if
15: end if

```

Runtime cost and local updates. Let $C_t = |\mathcal{H}_t| + |\mathcal{K}_t|$ denote the number of geometric regions evaluated at step t , and let s_c denote the number of coordinates used by region c . Computing the complete margin vector requires

$$O\left(\sum_{c=1}^{C_t} s_c\right) \leq O(C_t d) \quad (14)$$

operations. Each region uses only a subset of the execution relations, so margin evaluation scales with the active regions and the coordinates they select.

Observations from admitted tool executions or trusted user interactions update only the relations linked to the affected values, sources, authorization conditions, and state facts. Before evaluating the next proposed external effect, DAFR constructs its action–evidence point and reevaluates every applicable margin. Local relation updates and complete margin reevaluation limit computation to the active regions and their selected coordinates while preserving a full feasibility check for each proposal.

Evaluation

Experimental Setup

AgentDojo v1.2.2 is the primary benchmark and contains the Banking, Slack, Travel, and Workspace suites (Debenedetti et al. 2024). We evaluate DeepSeek V4 Flash, GPT-5.4-mini, and Claude Haiku 4.5 on the same 71 clean and 585 attacked cases for each planner. The comparison includes DAFR and eight baselines. Figure 2 presents six representative methods, and the supplement reports the complete results. Component ablations evaluate evidence projection, action–evidence lifting, geometric constraints, and decision feedback.

We further evaluate DAFR on 204 ASB cases using DeepSeek V4 Flash (Zhang et al. 2025). AgentDojo reports clean utility, attack utility, and attack success rate (ASR), while ASB reports utility, task success, and ASR.

DAFR is placed at the execution boundary, after the planner proposes a concrete tool call and before the trusted runtime performs its external effect. Each comparison keeps the

benchmark cases, planner model, registered tools, and tool schemas fixed. Under this setup, DAFR contributes the feasibility decision and relation-specific revision feedback for the proposed call, while the benchmark task and planning environment remain unchanged.

Main Results on AgentDojo

Figure 2 reports the AgentDojo comparison across three planner models. With DeepSeek V4 Flash, DAFR achieves 87.5% clean utility and 81.4% attack utility with 0% observed ASR. DAFR also records 0% observed ASR with GPT-5.4-mini and Claude Haiku 4.5, while retaining 59.5% and 68.4% attack utility, respectively.

PIGuard is the only other representative method that reaches 0% observed ASR with all three planners. At this security level, DAFR improves attack utility over PIGuard by 38.9, 25.1, and 30.2 percentage points on DeepSeek, GPT, and Claude, respectively. The feasible-region decision preserves more task progress under attack while maintaining zero observed attack success.

Results on Agent Security Benchmark

Table 1 reports results on 204 ASB cases with DeepSeek V4 Flash. DAFR achieves 86.8% utility and 73.5% task success with 0% observed ASR.

Method	Utility	Task Succ.	ASR
No Defense	88.7	77.9	90.2
Delimiter	87.3	77.0	86.3
Instruction Prevention	88.0	77.5	39.7
Observation Sandwich	88.0	77.0	27.9
TS-Flow	85.5	73.5	19.1
Progent	86.0	73.0	3.4
DRIFT	85.8	73.0	23.0
DAFR	86.8	73.5	0.0

Table 1: Results on 204 ASB cases with DeepSeek V4 Flash. Values are percentages.

Compared with no defense, DAFR reduces ASR from 90.2% to 0%, with a 1.9-point difference in utility. Among the evaluated methods, DAFR is the only one that reaches 0% observed ASR. Relative to Progent, the strongest competing method by ASR, DAFR gains 0.8 points in utility and 0.5 points in task success and reduces ASR from 3.4% to 0%. The ASB results extend DAFR’s security–utility balance beyond AgentDojo to a distinct tool-use benchmark.

Ablation and Geometric Analysis

Table 2 reports the full AgentDojo ablation with DeepSeek V4 Flash: each configuration uses the same 71 clean and 585 attacked cases across four suites, with suite-macro-averaged scores. Removing evidence projection lowers clean utility from 87.5% to 79.2% while retaining 81.8% attack utility and 0% observed ASR. Removing action–evidence lifting raises clean and attack utility to 91.5% and 85.3% but introduces a 0.7% observed ASR.

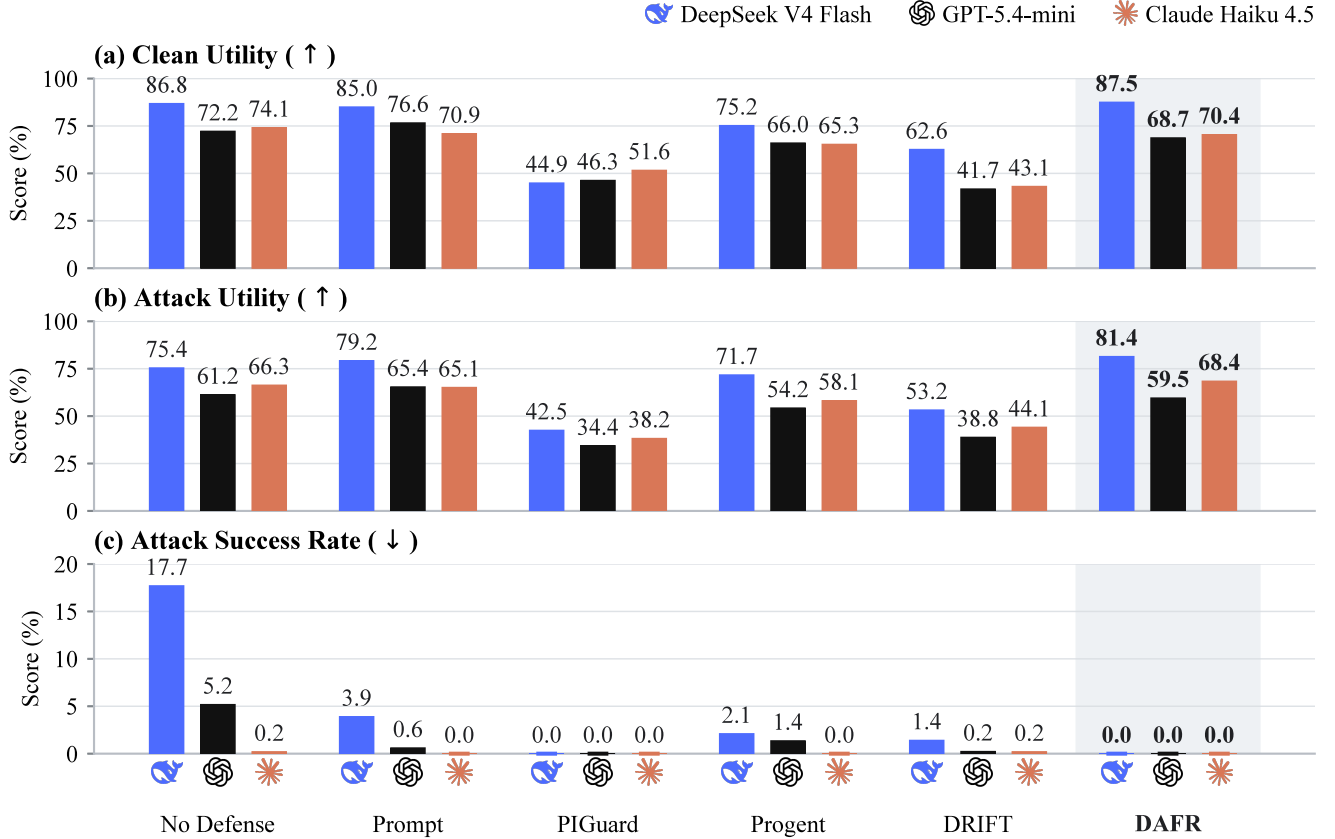


Figure 2: AgentDojo clean utility, attack utility, and attack success rate across six representative methods and three planner models. Values are percentages; arrows indicate the preferred direction. The supplement reports all nine evaluated methods.

Removing geometric constraints produces the largest security change in the ablation: observed ASR rises from 0% to 16.7%, with attack utility at 81.3%. Removing decision and feedback lowers clean and attack utility to 79.4% and 80.0% and introduces a 0.8% observed ASR. The ablation identifies geometric constraints as DAFR’s core security mechanism, with evidence projection and decision feedback preserving task utility.

Configuration	Clean	Attack	ASR
DAFR	87.5	81.4	0.0
w/o Evidence Projection	79.2	81.8	0.0
w/o Action–Evidence Lifting	91.5	85.3	0.7
w/o Geometric Constraints	90.2	81.3	16.7
w/o Decision & Feedback	79.4	80.0	0.8

Table 2: Component ablation on AgentDojo. Values are percentages.

Signed-margin behavior. Table 3 summarizes the overall signed margins of 9,083 AgentDojo tool calls across three planner models. Because DAFR admits a call exactly when $M_t(z_t) \geq 0$, the analysis reports the magnitude of the

raw signed slack produced by the active constraints. Mean margins are 0.712–0.750 for admitted calls and -0.368 to -0.214 for rejected calls. For each rejected call, the active condition attaining the minimum margin identifies the prerequisite, argument, source, authorization condition, or joint-risk relation used for targeted feedback.

Planner	Calls	$\bar{M}_{\text{adm}}$	$\bar{M}_{\text{rej}}$
DeepSeek V4 Flash	3947	0.712	-0.368
GPT-5.4-mini	2916	0.748	-0.214
Claude Haiku 4.5	2220	0.750	-0.289

Table 3: Mean signed margins for admitted and rejected AgentDojo calls.

Related Work

Prompt-injection attacks and defenses. Indirect prompt injection allows untrusted observations to redirect tool-using agents, as shown by early attacks and later benchmarks (Greshake et al. 2023; Liu et al. 2023; Zhan et al. 2024; Debenedetti et al. 2024). Context-level defenses preserve instruction boundaries, prioritize trusted instructions, or sepa-

rate instructions from data (Hines et al. 2024; Wallace et al. 2024; Chen et al. 2025). PIGuard detects injected content, Task Shield checks task alignment, and MELON compares normal execution with masked re-execution (Li et al. 2025b; Jia et al. 2025; Zhu et al. 2025).

Runtime controls for tool-using agents. AgentSpec and ShieldAgent express runtime requirements through explicit safety policies (Wang, Poskitt, and Sun 2025; Chen, Kang, and Li 2025). Progent enforces privileges over tool names and arguments, and DRIFT checks deviations from a task-grounded function trajectory (Shi et al. 2025; Li et al. 2025a). CaMeL separates control and data flows derived from a trusted request, and Fides enforces confidentiality and integrity labels (Debenedetti et al. 2025; Costa et al. 2025).

Tool-safety evaluation. ToolEmu and ToolSword study failures arising from consequential tool use (Ruan et al. 2024; Ye et al. 2024). AgentHarm and MobileSafetyBench extend evaluation to harmful multi-step behavior and mobile-device control, and MCPTox studies tool poisoning in MCP servers (Andriushchenko et al. 2025; Lee et al. 2026; Wang et al. 2026). Our evaluation uses AgentDojo and ASB to measure task utility and attack-induced tool effects.

Geometric safety formulations. Action shielding restricts decisions to actions that satisfy formal safety specifications (Alshiekh et al. 2018). GIF uses model Jacobians and local output geometry to bound information flow inside language models (Storek et al. 2026). DAFR instead places explicit action–evidence relations at the execution boundary. State-conditioned halfspaces and joint-risk regions combine task support, grounding, provenance, authorization, effects, and state in one feasibility decision, and active margins provide relation-specific revision targets.

Limitations

DAFR is evaluated on AgentDojo and ASB with three planner models, covering multiple tool suites, execution patterns, and risk categories. This scope establishes its behavior under two complementary benchmark protocols. Evaluation in additional domains can characterize its operation with other tools and execution settings.

DAFR derives support from tool outputs, environment observations, and trusted user interactions. If the required evidence, authorization, or state condition has not been established, the call remains outside the feasible region and DAFR requests the admissible support or blocks execution. Missing evidence is thus represented as an explicit execution state rather than treated as implicit approval.

New tools integrate by mapping their schema fields, behavioral effects, and authorization requirements to the shared execution roles. A tool with a new behavior or control structure extends the role map with the corresponding relation and policy template.

Conclusion

Tool calls can be well formed and relevant to a task yet lack the evidence, authorization, or state support required

for execution. DAFR addresses this gap with a deterministic action–evidence representation and a dynamic feasible region formed by policy-specified halfspaces and joint-risk constraints. Calls inside the region execute; calls outside it are blocked or returned with relation-specific feedback.

On AgentDojo and Agent Security Benchmark, DAFR reaches 0% observed attack success while retaining task utility across tool environments and planner models. The ablation identifies geometric constraints as the core security mechanism, and the signed margins connect rejected calls to their limiting execution relations. Together, these results establish dynamic feasible regions as a unified execution-layer formulation that turns task evidence, authorization, and state into auditable tool-action decisions.

References

- Alshiekh, M.; Bloem, R.; Ehlers, R.; Könighofer, B.; Niekum, S.; and Topcu, U. 2018. Safe Reinforcement Learning via Shielding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32.
- Andriushchenko, M.; Souly, A.; Dziemian, M.; Duenas, D.; Lin, M.; Wang, J.; Hendrycks, D.; Zou, A.; Kolter, Z.; Fredrikson, M.; Winsor, E.; Wynne, J.; Gal, Y.; and Davies, X. 2025. AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents. In *The Thirteenth International Conference on Learning Representations*.
- Chen, S.; Piet, J.; Sitawarin, C.; and Wagner, D. 2025. StruQ: Defending Against Prompt Injection with Structured Queries. In *34th USENIX Security Symposium*, 2383–2400.
- Chen, Z.; Kang, M.; and Li, B. 2025. ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 8313–8344. PMLR.
- Costa, M.; Köpf, B.; Kolluri, A.; Pavard, A.; Russinovich, M.; Salem, A.; Tople, S.; Wutschitz, L.; and Zanella-Béguelin, S. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643.
- Debenedetti, E.; Shumailov, I.; Fan, T.; Hayes, J.; Carlini, N.; Fabian, D.; Kern, C.; Shi, C.; Terzis, A.; and Tramèr, F. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813.
- Debenedetti, E.; Zhang, J.; Balunovic, M.; Beurer-Kellner, L.; Fischer, M.; and Tramèr, F. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems*, volume 37, 82895–82920.
- Greshake, K.; Abdelnabi, S.; Mishra, S.; Endres, C.; Holz, T.; and Fritz, M. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173.
- Hines, K.; Lopez, G.; Hall, M.; Zarfati, F.; Zunger, Y.; and Kiciman, E. 2024. Defending Against Indirect Prompt Injection Attacks With Spotlighting. arXiv:2403.14720.
- Jia, F.; Wu, T.; Qin, X.; and Squicciarini, A. 2025. The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents. In *Proceedings of the*

63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 29680–29697.

Lee, J.; Hahm, D.; Choi, J. S.; Knox, W. B.; and Lee, K. 2026. MobileSafetyBench: Evaluating Safety of Autonomous Agents in Mobile Device Control. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 37565–37573.

Li, H.; Liu, X.; Chiu, H.-C.; Li, D.; Zhang, N.; and Xiao, C. 2025a. DRIFT: Dynamic Rule-Based Defense with Injection Isolation for Securing LLM Agents. In *Advances in Neural Information Processing Systems*, volume 38, 83262–83290.

Li, H.; Liu, X.; Zhang, N.; and Xiao, C. 2025b. PIGuard: Prompt Injection Guardrail via Mitigating Overdefense for Free. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 30420–30437.

Li, H.; Wen, R.; Shi, S.; Zhang, N.; and Xiao, C. 2026. AgentDyn: A Dynamic Open-Ended Benchmark for Evaluating Prompt Injection Attacks of Real-World Agent Security System. arXiv:2602.03117.

Liao, Z.; Mo, L.; Xu, C.; Kang, M.; Zhang, J.; Xiao, C.; Tian, Y.; Li, B.; and Sun, H. 2025. EIA: Environmental Injection Attack on Generalist Web Agents for Privacy Leakage. In *The Thirteenth International Conference on Learning Representations*.

Liu, Y.; Deng, G.; Li, Y.; Wang, K.; Wang, Z.; Wang, X.; Zhang, T.; Liu, Y.; Wang, H.; Zheng, Y.; and Liu, Y. 2023. Prompt Injection Attack against LLM-Integrated Applications. arXiv:2306.05499.

Ma, X.; Li, T.; Xiao, C.; Yu, Z.; Zhang, N.; and Vorobeychik, Y. 2026. AutoDojo: Adaptive Attacks Expose Superficial Defenses and User-Underspecification Limits in LLM Agents. arXiv:2606.15057.

Ruan, Y.; Dong, H.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C. J.; and Hashimoto, T. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *The Twelfth International Conference on Learning Representations*.

Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, volume 36.

Shi, T.; He, J.; Wang, Z.; Wu, L.; Li, H.; Guo, W.; and Song, D. 2025. Progent: Programmable Privilege Control for LLM Agents. arXiv:2504.11703.

Storek, A.; Holzer, N.; Zhang, Z.; and Jana, S. 2026. GIF: Locally Sound Geometric Information Flow Control for LLMs. arXiv:2606.23277.

Wallace, E.; Xiao, K.; Leike, R.; Weng, L.; Heidecke, J.; and Beutel, A. 2024. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. arXiv:2404.13208.

Wang, H.; Poskitt, C. M.; and Sun, J. 2025. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. arXiv:2503.18666.

Wang, Z.; Gao, Y.; Wang, Y.; Liu, S.; Sun, H.; Cheng, H.; Shi, G.; Du, H.; and Li, X. 2026. MCPTox: A Benchmark for Tool Poisoning on Real-World MCP Servers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 35811–35819.

Wang, Z.; Siu, V.; Ye, Z.; Shi, T.; Nie, Y.; Zhao, X.; Wang, C.; Guo, W.; and Song, D. 2025. AGENTVIGIL: Automatic Black-Box Red-teaming for Indirect Prompt Injection against LLM Agents. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 23159–23172.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*.

Ye, J.; Li, S.; Li, G.; Huang, C.; Gao, S.; Wu, Y.; Zhang, Q.; Gui, T.; and Huang, X. 2024. ToolSword: Unveiling Safety Issues of Large Language Models in Tool Learning Across Three Stages. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2181–2211.

Zhan, Q.; Liang, Z.; Ying, Z.; and Kang, D. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, 10471–10506.

Zhang, H.; Huang, J.; Mei, K.; Yao, Y.; Wang, Z.; Zhan, C.; Wang, H.; and Zhang, Y. 2025. Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents. In *The Thirteenth International Conference on Learning Representations*.

Zhu, K.; Yang, X.; Wang, J.; Guo, W.; and Wang, W. Y. 2025. MELON: Provable Defense Against Indirect Prompt Injection Attacks in AI Agents. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 80310–80329. PMLR.